

Plan de Testing - Proyecto Aurora

Universidad Tecnológica Nacional (UTN) - Facultad Regional Córdoba (FRC)

Carrera: Ingeniería en Sistemas de Información

Asignatura: Proyecto Final (Curso 5K3) - Año 2026

Docentes: Ing. Sergio Quinteros, Mgter. Ing. Virginia Santos

Legajo	Apellido, Nombres	Email de Contacto
83009	Romero Plaza, Mateo	mateoromeroplaza12@gmail.com
85246	Kilic Aslan, Haik Martín	kilichaik@gmail.com
86537	Rodeyro Contarino, Nicolás	nicolasrodeyro@gmail.com
86291	Maldonado Gómez, Jeremías Daniel	jmaldonadogomez21@gmail.com
89866	Escudero, Octavio	octavioescudero448@gmail.com

Historial de Revisión

Versión	Fecha	Autor	Descripción del Cambio
1.0	24/05/2026	Equipo Aurora	Creación inicial del documento definiendo la estrategia de calidad y verificación. Incorporación de pruebas de IA, hardware, seguridad y UX.

1. Introducción

El presente **Plan de Testing** define la estrategia integral de verificación de calidad del sistema Aurora antes de la escritura masiva de código. Al tratarse de un sistema domiciliario orientado al acompañamiento y monitoreo de pacientes con deterioro cognitivo, la tolerancia a fallos en módulos de alerta temprana, monitoreo biométrico y, de forma crítica, la interacción cognitiva a través de Inteligencia Artificial (IA) es mínima. Este plan establece las pautas para asegurar que el sistema procesa, notifica, interactúa y responde a eventos de riesgo y rutinas con máxima precisión, seguridad (cumplimiento normativo) y confiabilidad, garantizando un entorno seguro, proactivo y empático.

2. Audiencia

Este documento está diseñado principalmente para dos grupos de interés:

- **El Equipo de Desarrollo:** Como referencia operativa obligatoria durante todos los sprints del ciclo de vida del proyecto para la elaboración y ejecución de pruebas.
- **El Cuerpo Docente:** Para facilitar la evaluación de la cobertura real, la viabilidad técnica y el aseguramiento de la calidad del sistema implementado.

3. Estrategia y Niveles de Testing

Se aplicará un enfoque de pruebas integral en múltiples capas para aislar errores tempranamente, verificando desde las reglas de negocio hasta las interacciones complejas de IA, integraciones IoT y cumplimiento de normativas de seguridad, enfocado en las interfaces del usuario (VUI y Móvil).

Nivel / Área de Testing	Objetivo en Aurora	Herramientas / Métodos Propuestos	Ejemplos de Casos Clave
Unitario	Validar el funcionamiento aislado de reglas de negocio centrales sin dependencias externas (bases de datos o hardware).	Jest / PyTest / JUnit (según stack)	• Algoritmo de clasificación de riesgo de eventos. • Cálculo de omisiones de rutinas y

Nivel / Área de Testing	Objetivo en Aurora	Herramientas / Métodos Propuestos	Ejemplos de Casos Clave
			medicación en el tiempo estipulado.
Integración	Asegurar la correcta comunicación entre los microservicios, la base de datos y la recepción de datos de la pulsera IoT.	Postman / Supertest / Testcontainers	Interacción del backend con la API de mensajería externa (alertas).
End-to-End (E2E)	Verificar el flujo de usuario completo, desde que ocurre un evento en el ambiente físico (o app) hasta el reflejo en la interfaz.	Appium (App Móvil) / Cypress (Web)	- Flujo: Detección de salida de "Zona Segura" -> Clasificación -> Notificación Push al Cuidador. - Flujo de configuración de perfil de paciente e historial biográfico.
Inteligencia Artificial (IA) y RAG	Garantizar la precisión, relevancia y seguridad de las interacciones basadas en modelos LLM y memoria contextual (RAG).	Prompt Evaluation Frameworks / Pruebas de validación humana en circuito	Precisión RAG: Verificar que la recuperación de recuerdos (ej: nombre de familiares) sea exacta. Prevención de Alucinaciones

Nivel / Área de Testing	Objetivo en Aurora	Herramientas / Métodos Propuestos	Ejemplos de Casos Clave
			s: Asegurar que el sistema no invente datos médicos ni se desvíe del flujo asistencial configurado ante consultas ambiguas del paciente.
Resiliencia (Hardware e IoT)	Verificar la robustez del sistema ante fallas de conectividad y asegurar la continuidad operativa en escenarios degradados.	Simuladores IoT / Chaos Testing básico	● Pérdida de Conexión/Batería: Respuesta del sistema ante la desconexión abrupta o batería baja de la pulsera IoT. ● Funcionamiento Degradado: Comprobar que el DOT y el módulo de RAG sigan funcionando (ej: recordatorios verbales) aunque los sensores biométricos estén caídos.

Nivel / Área de Testing	Objetivo en Aurora	Herramientas / Métodos Propuestos	Ejemplos de Casos Clave
Seguridad y Privacidad	Asegurar el cumplimiento estricto de la Ley 25.326 de Protección de Datos Personales y prevenir vulnerabilidades.	OWASP ZAP / Auditorías de seguridad manuales	• Pentesting en Endpoints: Pruebas de vulnerabilidad contra inyecciones y accesos no autorizados a historiales médicos. • Notificaciones Seguras: Validación de que las notificaciones push no exponen datos sensibles de forma visible.
Usabilidad y Accesibilidad (UX/UI)	Asegurar que las interfaces (Móvil y VUI) requieran un esfuerzo cognitivo mínimo para el paciente y sean intuitivas para el cuidador.	Evaluaciones Heurísticas / Pruebas de Usuario (Simuladas)	• VUI (DOT): Pruebas de comprensión de voz en distintos tonos, con ruido de fondo y pausas prolongadas (típicas del deterioro cognitivo). • App Cuidador: Validación heurística de

Nivel / Área de Testing	Objetivo en Aurora	Herramientas / Métodos Propuestos	Ejemplos de Casos Clave
			la aplicación móvil asegurando claridad en el dashboard de emergencias.

4. Criterios de Cobertura Mínima Aceptable

- **Cobertura de Código Backend:** Mínimo 80% de cobertura de líneas y branches lógicas (*Code Coverage*), especialmente en los motores de orquestación de rutinas y categorización de alertas.
- **Flujos Críticos (100%):** Las funciones que afecten la integridad física del paciente (ej. módulo de detección de riesgo alto/crítico, generación de alertas por caída, deambulación y omisión de medicación) deben contar con validación unitaria y E2E sin excepciones.
- **Pruebas de Usabilidad e Interfaz (VUI/App):** Ningún *release* puede contener cierres inesperados (*crashes*) durante flujos operativos normales o en la vista del panel de supervisión del cuidador.
- **Precisión IA:** Las respuestas generadas por IA deben mantener un umbral del 99% de precisión respecto a la memoria contextual registrada, sin alucinaciones médicas permitidas.

5. Tipos y Criticidad de Incidencias

Dado el nivel de dependencia asistencial que propicia el sistema, los defectos (*bugs*) se categorizarán bajo una matriz de impacto/urgencia rigurosa:

Nivel de Criticidad	Descripción del Impacto	Acuerdo de Resolución (SLA Interno)
Crítica (P1)	Compromete la integridad o asistencia del paciente.	Bloqueo inmediato. Resolución antes de

Nivel de Criticidad	Descripción del Impacto	Acuerdo de Resolución (SLA Interno)
	Fallas en la detección de caídas, omisión de envío de alertas a cuidadores, alucinaciones médicas del motor RAG, brechas de seguridad (Ley 25.326) o caída total del servidor/DOT.	fusionar la rama o finalizar el sprint.
Alta (P2)	Falla en un requerimiento funcional clave sin riesgo de vida inmediato. Ej: no registrar correctamente una respuesta ante estimulación cognitiva o desconexión no notificada de la pulsera IoT. Inexactitud en datos biográficos (IA).	Resolución obligatoria dentro del sprint activo.
Media (P3)	Errores en la visualización del historial, reportes desfasados visualmente, fallos menores de usabilidad o fallos en canales secundarios de notificación.	Se prioriza en el Backlog para el siguiente sprint.
Baja (P4)	Desalineamientos de UI, errores ortográficos en el dashboard, o sugerencias de optimización técnica de la latencia en las respuestas del sistema.	Resolución paulatina en tareas de mantenimiento.

6. Ejecución y Revisión de Resultados

El proceso de testing no será una actividad al final del desarrollo, sino un control continuo:

- **Ejecución Automática:** Los test unitarios, de integración y evaluaciones automatizadas

del pipeline RAG se integrarán en una canalización CI/CD (*Continuous Integration / Continuous Deployment*). Se ejecutarán automáticamente al generarse un nuevo *Pull Request* hacia la rama de desarrollo.

- **Revisión por Pares (Code Review):** Un desarrollador diferente al que escribió la funcionalidad deberá revisar el PR, verificando que los nuevos casos de prueba presentados cuenten con los criterios "Dado, Cuando, Entonces" (*Given, When, Then*). Se auditarán de manera especial los cambios en las reglas de seguridad y prompts de IA.
- **Verificación de Aceptación (QA y UX):** Durante la fase de *Sprint Review*, el equipo en su conjunto, junto con el Product Owner/Docentes, verificará la ejecución de los escenarios E2E más representativos del ciclo, simulando interacciones reales tanto desde la app del cuidador como desde la interfaz de voz (DOT) [cite: 504].